Proyecto tareaApis

Explicación de cada archivo y del código

App Android hecha con **Kotlin + Jetpack Compose** que consume tres APIs públicas y las muestra en tres pestañas: **Pokémon** (PokeAPI), **Rick y Morty** (Rick and Morty API) y **Marvel** (Superhero API). Cada pestaña tiene un buscador, una cuadrícula con imágenes y, al tocar un elemento, una pantalla de detalle.

Este documento explica: (1) cómo está organizado el proyecto en capas, (2) qué hay exactamente en cada archivo, (3) el recorrido completo de los datos usando Rick y Morty como ejemplo, y (4) de dónde sale cada imagen.

Todas las rutas de este documento están dentro de `app/src/main/java/com/example/tareaapis/`.

1. La idea general: capas

El código está dividido en carpetas, y cada carpeta tiene **una sola responsabilidad**. Los datos viajan siempre en el mismo orden, desde internet hasta la pantalla:

```
Internet (JSON)
     |
     v
data/api         -> Retrofit hace la petición HTTP
     |
     v
data/dto         -> el JSON se convierte en objetos Kotlin (copia exacta del JSON)
     |
     v
data/mapper      -> el DTO se limpia y se traduce al modelo de la app
     |
     v
domain/model     -> modelo "limpio" que usa la app (Pokemon, Hero, RickMortyCharacter)
     |
     v
data/repository  -> junta todo, guarda en memoria y maneja los errores
     |
     v
ui/viewmodel     -> guarda el estado de la pantalla (cargando / error / datos)
     |
     v
ui/screens + ui/components  -> Compose dibuja la pantalla
```

Carpeta	Responsabilidad	¿Sabe de la pantalla?
data/api	Define las URLs y los endpoints de cada API.	No
data/dto	Clases que replican el JSON tal como llega.	No
data/mapper	Convierte DTO a modelo (limpia datos, traduce textos).	No
data/repository	Pide los datos, los guarda en memoria y convierte los errores en mensajes.	No
domain/model	Los modelos que realmente usa la app.	No
ui/viewmodel	Estado de cada pantalla y lógica del buscador.	Sí, pero sin dibujar nada
ui/components	Piezas de pantalla reutilizables (tarjeta, buscador, barras...).	Sí
ui/screens	Una pantalla por API: aquí se decide cómo se ve cada una.	Sí
ui/theme	Colores y tipografía de la app (viene de la plantilla de Android Studio).	Sí

Por qué sirve esta división: si mañana una API cambia su JSON, solo se tocan *dto* y *mapper*; la pantalla ni se entera. Si se quiere cambiar cómo se ve una tarjeta, solo se toca *screens*.

2. Árbol de archivos

```
tareaapis/
|-- MainActivity.kt
|-- data/
|   |-- api/
|   |   |-- ApiClient.kt
|   |   |-- PokeApi.kt
|   |   |-- RickAndMortyApi.kt
|   |   `-- MarvelApi.kt
|   |-- dto/
|   |   |-- PokemonDto.kt
|   |   |-- RickMortyDto.kt
```

```
|   |   `-- MarvelDto.kt
|   |-- mapper/
|   |   |-- MapperUtils.kt
|   |   |-- PokemonMapper.kt
|   |   |-- RickMortyMapper.kt
|   |   `-- MarvelMapper.kt
|   `-- repository/
|       |-- SafeApiCall.kt
|       |-- PokemonRepository.kt
|       |-- RickMortyRepository.kt
|       `-- MarvelRepository.kt
|-- domain/model/
|   `-- Models.kt
`-- ui/
    |-- viewmodel/
    |   |-- UiState.kt
    |   |-- CatalogViewModel.kt
    |   |-- PokemonViewModel.kt
    |   |-- RickMortyViewModel.kt
    |   `-- MarvelViewModel.kt
    |-- components/
    |   |-- Catalog.kt
    |   |-- Common.kt
    |   `-- Detail.kt
    |-- screens/
    |   |-- PokemonScreen.kt
    |   |-- RickMortyScreen.kt
    |   `-- MarvelScreen.kt
    `-- theme/  (Color.kt, Theme.kt, Type.kt)
```

3. Qué hay en cada archivo

3.1 MainActivity.kt (punto de entrada)

Es lo primero que Android ejecuta. Dibuja la barra superior, la barra de navegación de abajo con las 3 pestañas y muestra la pantalla que corresponda.

- **enum Tab**: lista las pestañas (POKEMON, RICK_MORTY, MARVEL), cada una con su emoji, etiqueta y título.
- **App()**: guarda la pestaña elegida con `rememberSaveable` (así no se pierde al girar el teléfono) y con un `when` muestra `PokemonScreen`, `RickMortyScreen` o `MarvelScreen`.

3.2 data/api (las peticiones a internet)

Archivo	Qué contiene
ApiClient.kt	Un **object** (existe una sola copia en toda la app) que crea los clientes de red. Tiene las 3 URLs base (pokeapi.co, rickandmortyapi.com y akabab.github.io/superhero-api), un **OkHttpClient** compartido (con tiempos máximos de espera y un log de las peticiones) y la función *create*, que arma un Retrofit con Gson (el que convierte JSON a objetos). Expone *pokeApi*, *rickAndMortyApi* y *marvelApi*, creados solo la primera vez que se usan (`by lazy`).
PokeApi.kt	Interface de Retrofit. *getPokemonList* (lista con nombre y url), *getPokemon(id)* (datos, estadísticas, imágenes) y *getSpecies(id)* (descripción, categoría, generación, hábitat).
RickAndMortyApi.kt	Interface de Retrofit. *getCharacters(page)* (una página de 20), *getCharacter(id)* (uno solo) y *getCharactersByIds("1,2,3")* (varios en una sola petición).
MarvelApi.kt	Interface de Retrofit. *getAllHeroes()* trae todos los héroes en un solo archivo `all.json` y *getHero(id)* trae uno.

Una **interface de Retrofit** no tiene código: solo describes la petición con anotaciones y Retrofit genera el código por ti. Ejemplo real del proyecto:

```
interface RickAndMortyApi {
    @GET("character")                       // GET https://rickandmortyapi.com/api/character
    suspend fun getCharacters(@Query("page") page: Int): RickMortyPageDto   // ?page=1

    @GET("character/{ids}")                 // {ids} se reemplaza por lo que le pases
    suspend fun getCharactersByIds(@Path("ids") ids: String): List<RickMortyCharacterDto>
}
```

`suspend` significa que la función puede "esperar" a internet sin congelar la pantalla (corrutinas de Kotlin).

3.3 data/dto (el JSON como clases Kotlin)

Un **DTO** (Data Transfer Object) es una clase cuyos campos tienen **el mismo nombre que el JSON**. Gson llena la clase automáticamente. Solo se declaran los campos que la app usa.

Archivo	Qué contiene
PokemonDto.kt	PokemonListDto, PokemonDto (id, nombre, altura, peso, sprites, tipos, habilidades, stats, movimientos), SpritesDto/ArtworkDto (las URLs de las imágenes), PokemonSpeciesDto y sus auxiliares. `@SerializedName("front_default")` se usa cuando el nombre del JSON tiene guion bajo y en Kotlin se escribe en camelCase (frontDefault).
RickMortyDto.kt	RickMortyPageDto (info + results), RickMortyInfoDto (count, pages), RickMortyCharacterDto (id, name, status, species, type, gender, origin, location, **image**, episode) y RickMortyPlaceDto.

Archivo	Qué contiene
MarvelDto.kt	SuperheroDto con sus bloques powerstats, appearance, biography, work, connections e **images** (xs, sm, md, lg: cuatro tamaños de la misma foto).

3.4 data/mapper (limpiar y traducir)

La API manda datos "crudos" (inglés, valores como "-" o "unknown" cuando no hay dato, altura en decímetros...). El mapper los convierte al modelo de la app.

Archivo	Qué hace
MapperUtils.kt	Funciones auxiliares compartidas: *idFromUrl()* (de ".../pokemon/25/" saca 25), *prettify()* ("special-attack" pasa a "Special attack"), *cleanOrNull()* (convierte "-", "unknown", "none"... en null para no mostrarlos) y *translateGender()* (male pasa a Masculino, etc.).
PokemonMapper.kt	*toPokemonSummary()*: para la lista arma la URL de la imagen con el id. *toDomain()*: convierte altura/peso a metros/kg, elige la descripción en español (si no hay, en inglés), traduce stats y hábitats con diccionarios y elige la imagen oficial.
RickMortyMapper.kt	Convierte el estado ("Alive"/"Dead"/otro) en el enum CharacterStatus, limpia tipo/origen/ubicación con cleanOrNull, traduce el género y cuenta los episodios (`episode.size`). La imagen se copia tal cual desde `image`.
MarvelMapper.kt	Convierte alineación (good/bad), arma la lista de poderes con nombre en español, se queda con las medidas en cm/kg y limpia todos los textos vacíos.

Ejemplo (RickMortyMapper): `fun RickMortyCharacterDto.toDomain()` es una *función de extensión*: se escribe como si el DTO tuviera ese método, así que se usa `dto.toDomain()`.

3.5 domain/model/Models.kt (los modelos de la app)

Un solo archivo con las clases que usan las pantallas. Son más simples que los DTO y ya vienen listas para mostrar (los campos opcionales son `String?`: si son null, la pantalla no dibuja esa fila).

- **Stat**: nombre + valor (la usan Pokémon y Marvel para las barras).
- **PokemonSummary** (para la lista) y **Pokemon** (para el detalle).
- **CharacterStatus** (enum ALIVE/DEAD/UNKNOWN) y **RickMortyCharacter**.
- **HeroAlignment** (enum GOOD/BAD/NEUTRAL) y **Hero**.

3.6 data/repository (conseguir los datos)

Archivo	Qué hace
SafeApiCall.kt	Función *safeApiCall* que ejecuta cualquier petición dentro de un try/catch y devuelve un **Result** (éxito o fallo) con un mensaje entendible: 404 = no existe, 429 = demasiadas peticiones, 5xx = el servidor no responde, sin internet = "Sin conexión a internet". Así ningún error cierra la app.
PokemonRepository.kt	*getPokemonList()*: pide los 1025 Pokémon en una llamada y los guarda en memoria (cache). *getPokemon(id)*: pide el Pokémon **y** su especie al mismo tiempo con *async* y los junta.
RickMortyRepository.kt	*getCharacters()*: pide todos los personajes (ver sección 4) y los guarda en memoria. *getCharacter(id)*: busca en la lista ya cargada y solo si no está la pide a la API.
MarvelRepository.kt	*getHeroes()*: descarga todos los héroes, se queda con los de "Marvel Comics" y los ordena por nombre. *getHero(id)*: igual que en Rick y Morty.

3.7 ui/viewmodel (el estado de cada pantalla)

Archivo	Qué hace
UiState.kt	Interface sellada con 3 casos: **Loading** (cargando), **Success(data)** (llegó todo bien) y **Error(message)**. Incluye *toUiState()* que convierte un Result del repositorio en un UiState.
CatalogViewModel.kt	**El archivo más importante de la capa.** Es una clase genérica `CatalogViewModel<Item, Detail>` que sirve para las 3 APIs. Se encarga de: cargar la lista al iniciar, filtrarla con el buscador (*query*), y abrir/cerrar/reintentar el detalle de un elemento. Recibe 3 funciones: cómo cargar la lista, cómo cargar un detalle y cómo saber si un elemento coincide con el texto buscado.
PokemonViewModel.kt RickMortyViewModel.kt MarvelViewModel.kt	Cada uno es solo **unas pocas líneas**: heredan de CatalogViewModel y le pasan las 3 funciones de su repositorio. Por ejemplo, el de Pokémon busca por nombre o por número; el de Marvel por nombre de héroe o nombre real.

¿Por qué un ViewModel? Sobrevive a los giros de pantalla: si giras el teléfono, la lista y el texto del buscador no se pierden ni se vuelve a descargar nada. El estado se expone como **StateFlow** y la pantalla se vuelve a dibujar sola cuando cambia.

3.8 ui/components (piezas reutilizables)

Archivo	Qué contiene
Catalog.kt	**CatalogScreen**: la pantalla genérica que usan las 3 APIs (buscador arriba + cuadrícula con *LazyVerticalGrid*; al tocar algo muestra el detalle; el botón atrás del teléfono cierra el detalle). **CatalogCard**: la tarjeta con imagen arriba y texto abajo. La imagen se carga con **AsyncImage** de la librería Coil, que la descarga de la URL.
Common.kt	Piezas pequeñas: **StateContent** (decide si dibujar cargando, error o contenido según el UiState), LoadingBox, ErrorBox (con botón Reintentar), EmptyBox (sin resultados), SearchField, Tag (etiqueta de color) e InfoValue (valor grande con etiqueta).
Detail.kt	Piezas del detalle: **DetailLayout** (columna con scroll), **DetailHeader** (cabecera con degradado, imagen grande, nombre y etiquetas), **InfoRow** (fila de valores grandes), **PropertySection** (sección de "etiqueta: valor" que no se dibuja si todo es null), **PillSection** (píldoras de texto con botón Ver todos), **StatsSection** (barras de estadísticas) y SectionCard.

3.9 ui/screens (una pantalla por API)

Aquí solo se decide **cómo se ve cada API**, porque la lógica ya está en CatalogScreen. Cada archivo hace lo mismo: llama a *CatalogScreen* pasando cómo es la tarjeta y cómo es el detalle.

Archivo	Qué muestra
PokemonScreen.kt	Tarjeta con imagen y número (#0025). Detalle: color según el tipo, descripción, altura/peso, barras de stats, generación, hábitat, habilidades, imagen normal y variocolor, movimientos. Tiene el diccionario POKEMON_TYPES (nombre en español + color de cada tipo).
RickMortyScreen.kt	Tarjeta con imagen, especie y estado. Detalle: imagen, estado y género como etiquetas, especie y número de episodios, y sección con tipo, origen y última ubicación.
MarvelScreen.kt	Tarjeta con foto vertical. Detalle: bando (héroe/villano), poderes en barras, biografía, alias, apariencia, trabajo y conexiones.

3.10 ui/theme y archivos de configuración

- **ui/theme/ (Color.kt, Theme.kt, Type.kt)**: colores y tipografía de la app. Es la plantilla que crea Android Studio; no tiene lógica propia.
- **AndroidManifest.xml**: declara la **Activity** principal y el permiso `INTERNET`, sin el cual no se podría llamar a ninguna API.
- **app/build.gradle.kts** y **gradle/libs.versions.toml**: las librerías. **Retrofit** + **Gson** (peticiones y JSON), **OkHttp logging** (ver peticiones en Logcat), **Coil** (descargar imágenes), Compose + Material 3 (la interfaz) y ViewModel/Lifecycle.

4. Recorrido completo: Rick y Morty

Sigamos qué pasa desde que abres la pestaña hasta que ves un personaje en pantalla.

Paso 1: la pantalla crea el ViewModel

```
// RickMortyScreen.kt
@Composable
fun RickMortyScreen(modifier: Modifier = Modifier, viewModel: RickMortyViewModel = viewModel()) {
    CatalogScreen(
        viewModel = viewModel,
        searchPlaceholder = "Buscar personaje",
        itemId = { it.id },
        itemContent = { character, onClick -> CatalogCard(...) },          // cómo es la tarjeta
        detailContent = { character, onBack -> CharacterDetail(character, onBack) }  // cómo es el detalle
    )
}
```

`viewModel()` crea (o reutiliza) el RickMortyViewModel. Al crearse, el *init* de CatalogViewModel llama a *reloadItems()* y empieza la descarga.

Paso 2: el ViewModel le pide la lista al repositorio

```
// RickMortyViewModel.kt
class RickMortyViewModel(repository: RickMortyRepository = RickMortyRepository())
  : CatalogViewModel<RickMortyCharacter, RickMortyCharacter>(
    loadItems = repository::getCharacters,     // "así se carga la lista"
    loadDetail = repository::getCharacter,     // "así se carga un detalle"
    matches = { character, query -> character.name.contains(query, ignoreCase = true) }
)
```

Mientras llega la respuesta el estado es **Loading** y se ve el círculo de carga.

Paso 3: el repositorio hace solo 2 peticiones

```
// RickMortyRepository.kt
private suspend fun fetchAllCharacters(): List<RickMortyCharacterDto> {
    val total = api.getCharacters(page = 1).info.count          // petición 1: ¿cuántos hay? (826)
    return api.getCharactersByIds((1..total).joinToString(","))  // petición 2: "1,2,3,...,826"
}
```

Por qué solo 2: la API entrega 20 personajes por página, así que traer todos son 42 peticiones. La API bloquea con el error **429 (demasiadas peticiones)** tras unas 30 seguidas. Pidiendo por ids se traen los 826 en una sola respuesta.

Paso 4: el JSON se vuelve DTO y luego modelo

```
// JSON que manda la API (recortado)
{"id":1, "name":"Rick Sanchez", "status":"Alive", "species":"Human", "type":"",
 "gender":"Male", "origin":{"name":"Earth (C-137)"}, "image":"https://.../avatar/1.jpeg",
 "episode":["https://.../episode/1", "https://.../episode/2", ...]}

// Gson lo mete en RickMortyCharacterDto, y el mapper lo convierte:
fun RickMortyCharacterDto.toDomain() = RickMortyCharacter(
    imageUrl = image,                        // la imagen viene directo de la API
    status = when (status) { "Alive" -> ALIVE; "Dead" -> DEAD; else -> UNKNOWN },
    type = type.cleanOrNull(),               // "" pasa a null y no se muestra
    gender = gender.cleanOrNull()?.let(::translateGender),   // Male pasa a Masculino
    episodeCount = episode.size,             // cantidad de episodios
    ...
)
```

Paso 5: safeApiCall protege todo

El repositorio envuelve el trabajo en `safeApiCall { ... }`. Si algo falla (sin internet, 429, 500...), en vez de cerrar la app devuelve un **Result.failure** con un mensaje, que el ViewModel convierte en **UiState.Error**, y la pantalla muestra el mensaje con el botón Reintentar.

Paso 6: el buscador filtra al instante

```
// CatalogViewModel.kt
val items = combine(allItems, _query) { state, query ->
    if (state is UiState.Success && query.isNotBlank())
        UiState.Success(state.data.filter { matches(it, query.trim()) })  // filtra en memoria
    else state
}
```

Todos los personajes ya están en memoria, así que filtrar no hace ninguna petición: es instantáneo. `combine` vuelve a calcular la lista cada vez que cambia la lista original o el texto.

Paso 7: Compose dibuja y Coil descarga las imágenes

```
// Catalog.kt
AsyncImage(model = imageUrl, contentDescription = title, ...)
```

AsyncImage (Coil) recibe la URL que vino de la API, descarga la imagen en segundo plano, la guarda en caché y la muestra. La cuadrícula es perezosa (*LazyVerticalGrid*): solo dibuja y descarga las tarjetas que se ven en pantalla.

Paso 8: tocar una tarjeta abre el detalle

Al tocar, *openDetail(id)* pone el estado del detalle en Loading y llama a *repository.getCharacter(id)*. Como el personaje ya está en memoria, la respuesta es inmediata. CatalogScreen ve que el detalle ya no es null y dibuja *CharacterDetail* en lugar de la lista. El botón atrás llama a *closeDetail()* y se vuelve a la lista en la misma posición.

5. De dónde sale cada imagen

Pantalla	Imagen	Origen
Rick y Morty	Lista y detalle	Campo `image` del JSON de cada personaje (rickandmortyapi.com). Es 100% de la API.
Marvel	Lista (*images.md*) y detalle (*images.lg*)	Campo `images` del JSON de cada héroe (Superhero API). 100% de la API.
Pokémon	Detalle y variocolor	Campo `sprites.other.official-artwork` del JSON de PokeAPI. 100% de la API.
Pokémon	Lista (tarjetas)	El endpoint de lista solo devuelve nombre y url, **sin imagen**. Se arma la URL con el id apuntando al repositorio oficial de sprites de PokeAPI (`PokemonMapper.kt`, constante ARTWORK_URL). Traer la imagen real de cada uno obligaría a hacer 1025 peticiones.

6. Resumen de conceptos

Concepto	En una frase
Retrofit	Librería que convierte una interface con anotaciones en peticiones HTTP.
Gson	Convierte el JSON en objetos Kotlin (los DTO).
DTO	Copia fiel del JSON.
Mapper	Convierte DTO en modelo limpio (traduce, limpia, calcula).
Repository	Obtiene los datos, los guarda en memoria y controla los errores.
ViewModel	Guarda el estado de la pantalla y sobrevive a los giros.
StateFlow	Valor observable: cuando cambia, la pantalla se redibuja sola.
Corrutina / suspend	Forma de esperar a internet sin congelar la app.
Composable	Función que dibuja una parte de la pantalla.
Coil / AsyncImage	Descarga y muestra imágenes desde una URL.
Genéricos (CatalogViewModel / CatalogScreen)	Se escribe la lógica una sola vez y sirve para las 3 APIs.